Automated Code Grading: Recent Approaches

Author: Chu Nguyen Gia Khanh
Date: March 7, 2026

Important Notes

For the upcoming project on the automated grading of coding assignments, **Gemma 3 27B** (accessed via the Google AI Studio API) will be used as the main LLM. Based on practical availability and cost constraints, this model appears to be the most suitable option for the project.

Contents

1. Introduction
2. Recent Methods leveraging LLMs for automated code assessment
   2.1 StepGrade: Chain-of-Thought and Prompt Chaining
   2.2 CodEv - CoT and LLM Ensemble (Intra-model Ensemble)
   2.3 A Combination of Unit Testing, LLM and Machine Learning

References

1. Introduction

Automated grading of programming exercises has become an important problem in computer science education. Programming courses at university often enroll up to hundreds of students, and with weekly assignments, manual grading can be labour-intensive and time-consuming. As a result, researchers and educators have explored automated systems capable of evaluating code quickly and with reasonable accuracy.

This documentation presents several recent methods leveraging LLMs for automated code assessment, with the goal of identifying useful ideas for the upcoming project aimed at automating or assisting the grading of coding assignments.

2. Recent Methods Leveraging LLMs for Automated Code Assessment

2.1 StepGrade: Chain-of-Thought and Prompt Chaining

Chain-of-Thought (CoT) and prompt chaining were utilised in an automated framework called StepGrade to guide an LLM (specifically GPT-4) in evaluating code according to specific rubrics in several prompts [1].

The framework grades the submission using one prompt per rubric, resulting in the following three-step process:

- For the first step in the process, GPT-4 is given the assignment question (for context) and a code submission to be evaluated. Along with these is a prompt guiding the model to assess the functionality of the code.
- For later steps (steps 2 and 3), apart from the question, submission and prompt for the rubric, GPT-4 also receives the its previous response of all previous step to provide context-aware responses.

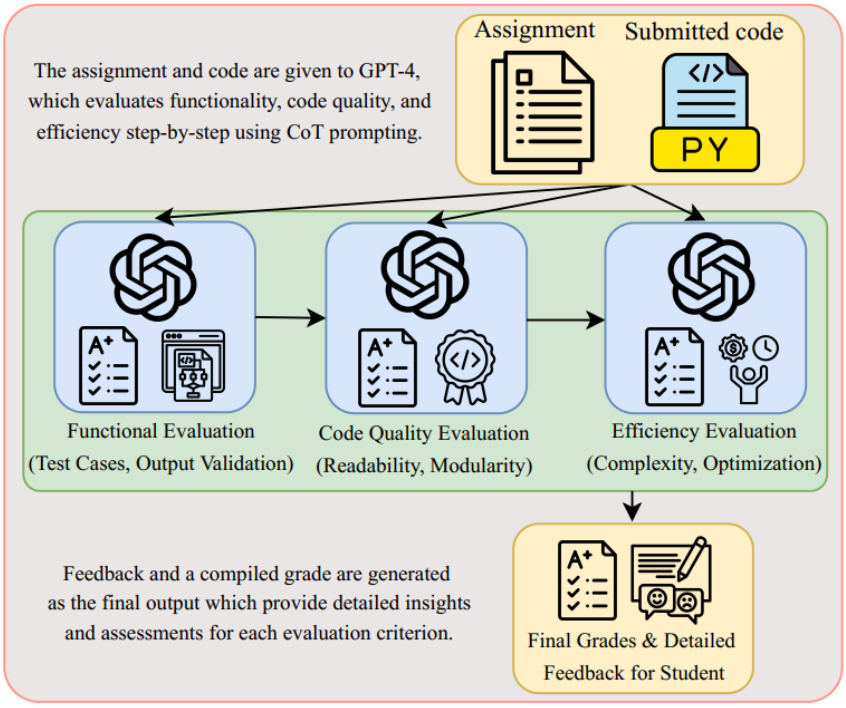

Fig 1. *The grading process in the StepGrade framework.*

The detailed prompt given to the model at each step is as follows:

Functionality

You are an expert programmer tasked with evaluating the functionality of a student's code submission for a programming assignment. Below is the assignment question and the student's code. Please follow these steps:

1. Analyze whether the code produces correct outputs for the given problem.
2. Test the code against a variety of standard and edge-case inputs (you may assume reasonable test cases for the problem).
3. Identify any logical errors, missing functionality, or unhandled edge cases.
4. Provide feedback that identifies specific issues and suggests improvements.

Finally, assign a grade (out of 10) for functionality based on how well the code meets the assignment's requirements.

Code quality

You are an expert programmer evaluating the quality of a student's code. Below is the assignment question, the student's code, and the functionality evaluation from the previous step. Please follow these steps:

1. Assess the readability of the code, including variable names, comments, and logical organization.
2. Evaluate whether the code follows best practices, such as modularity, avoiding redundancy, and proper use of structures.
3. Analyze the maintainability of the code—whether it would be easy for someone else to understand and modify.
4. Provide actionable feedback on how the student can improve the quality of their code.

Finally, assign a grade (out of 10) for code quality.

Algorithmic efficiency

You are an expert programmer evaluating the algorithmic efficiency of a student's code submission. Below is the assignment question, the student's code, and the responses from the functionality and code quality evaluations. Please follow these steps:

1. Analyze the time and space complexity of the code.
2. Identify any inefficiencies or performance bottlenecks in the implementation.
3. Suggest specific optimizations, such as improving algorithms, restructuring loops, or using more efficient data structures.
4. Provide constructive feedback on how the student can improve the efficiency of their code.

Finally, assign a grade (out of 10) for algorithmic efficiency.

Fig 2. *The detailed prompt at each step of the grading process.*

In the paper introducing the StepGrade framework, the approach achieved a Mean Absolute Error (MAE) of around 4.0% to 6.2% for each grading rubric relative to human grading on 30 Python programming assignments.

Overall, by utilising CoT and prompt chaining, the framework achieved reasonably accurate grading results compared to human. However, this paper was examining the framework using GPT-4, a large model, which will definitely not be available for the project. This indicates that with the currently available Gemma 3 27B API, the same workflow is likely to yields worse results.

2.2 CodEv - CoT and LLM Ensemble (Intra-model Ensemble)

CodEv is a framework that utilises CoT and LLM ensemble to grade code on several rubrics (correctness of output, code readability and functionality) [2]. The workflow can be summarised as follows:

- There is a CoT guided prompt that includes the problem statement, grading criteria and the student answers. This prompt is then be used to generate scores repeatedly (up to 20 times) from a single LLM.
- The mode of the scores generated is then selected as the final score for a student answer (Sampling and Voting).

$$\mathrm{MAE} = \frac{1}{m}\sum_{i=1}^{m}\left|\mathrm{Mode}\{\mathrm{Score}_{i,q}\}_{q=1}^{Q} - \mathrm{Score}_{i,\mathrm{human}}\right|$$

Fig 3. *The mode being chosen as the final score*

(m is the number of students, Q is the number of ensemble iterations for each student submission).

This framework is evaluated using four different LLMs: Smaller models include Llama 3.1 8B, Gemma 2 9B, and for larger models, Llama 3.1 70B and GPT-4o. The results were highly consistent.

For larger models, the final score MAE relative to human scoring (around 6.5%) was close to that of the StepGrade framework despite greater complexity. Nevertheless, for smaller models, the MAE was approximately double this number (about 12.5%).

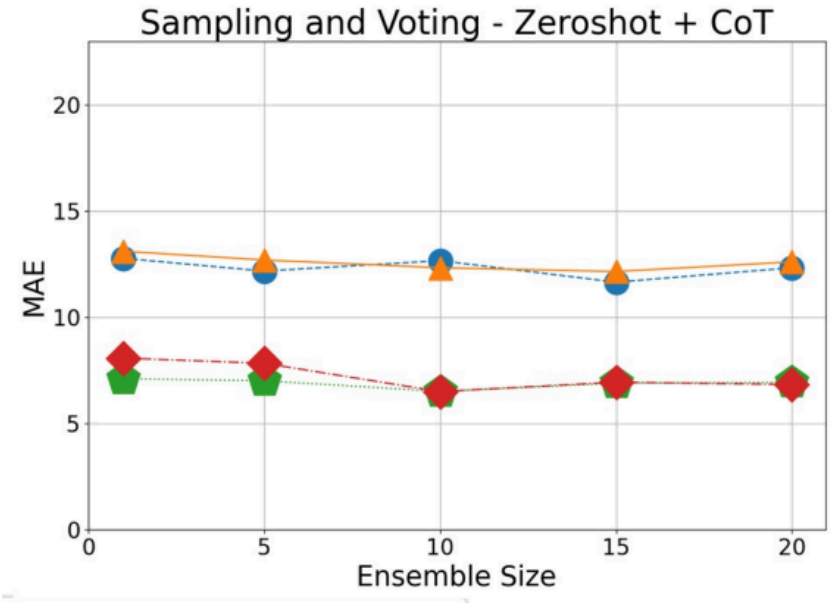

Fig 4. *MAE graphed as a function of the number of ensemble iteration.*

Model	Zero-Shot	CodEv	diff
Llama3.1 8B	17.22	12.66	-4.56
Gemma2 9B	20.61	12.32	-8.29
Llama3.1 70B	6.30	6.50	+0.20
GPT-4o	6.30	6.48	+0.18

Fig 5. *MAE comparison of the CodEv framework and Zero-shot approach without CoT.*

Overall, for larger models, CodEv performed reasonably well despite its added complexity, although its reported MAE was slightly worse than that of StepGrade. In contrast, the smaller models performed substantially worse. This is an important consideration for the upcoming project, since the use of Gemma 3 27B may place the system closer to a constrained setting than to the high-performance setting used in StepGrade. As a results, the project may produce a higher MAE than systems built on larger models using either StepGrade or CodEv framework (to be verified).

2.3 A Combination of Unit Testing, LLM and Machine Learning

Another workflow combines dynamic testing, LLM and machine learning [3]. Because the paper does not assign it a specific name, this document refers to it as LLM-AC (for LLM-Auto Correction) in the rest of this documentation for ease of referencing.

LLM-AC consists of three main modules:

- The Automatic Correction Module:
  - This module assesses the code's functionality by first compiling it, then testing it against predefined unit tests.
  - If the compilation or testing fails, the code is then given to GPT-4-Turbo to attempt a correction (where the prompt instructs the LLM to correct the code with minimal changes, keeping it close to original).
  - The corrected code is then re-evaluated with the same tests to ensure valid output (the attempt to correct is repeated until all tests are passed, up to 5 times).
  - If the code cannot be corrected, it is marked invalid; otherwise, it is passed through the next module for further evaluation along with the student submission.

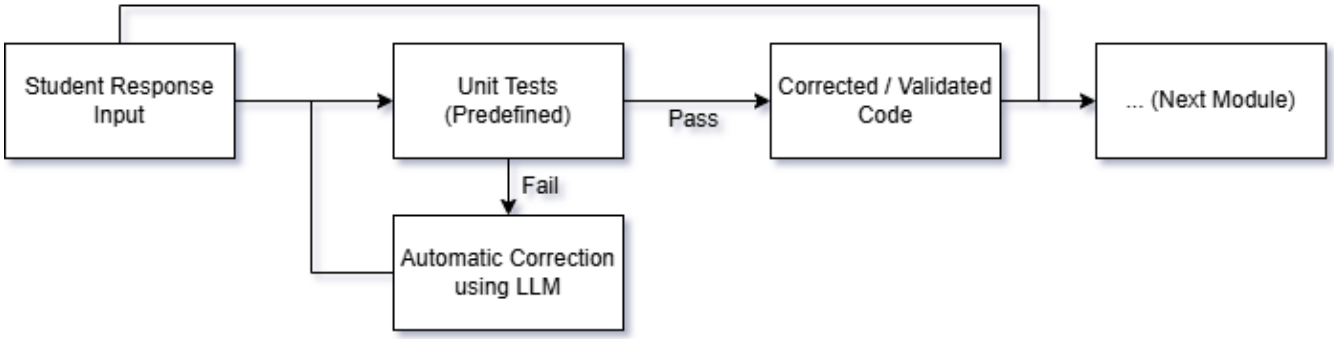

Fig 6. *The workflow of the Automatic Correction Module.*

- The Similarity and Evaluation Module:
  - In this phase, a modified version of Python SequenceMatcher is used to measure how the corrected code differs from the original code (by classifying them into correct, updated, added and removed lines).
  - Each line is also classified into logic, input or output role by analysing its Abstract Syntax Tree. With each role, the line is given a weight corresponding to their significance (3, 2, and 1

for logic, input and output respectively), called line importance.

- After classifying each line into different change types (correct, updated, added and removed) and applying weights, the ratio for each category is then calculated (However, for modified lines, an edit distance similarity score is calculated and integrated into its line importance to give more credit for less changed code).
- The ratios is then be used as input to the final module.

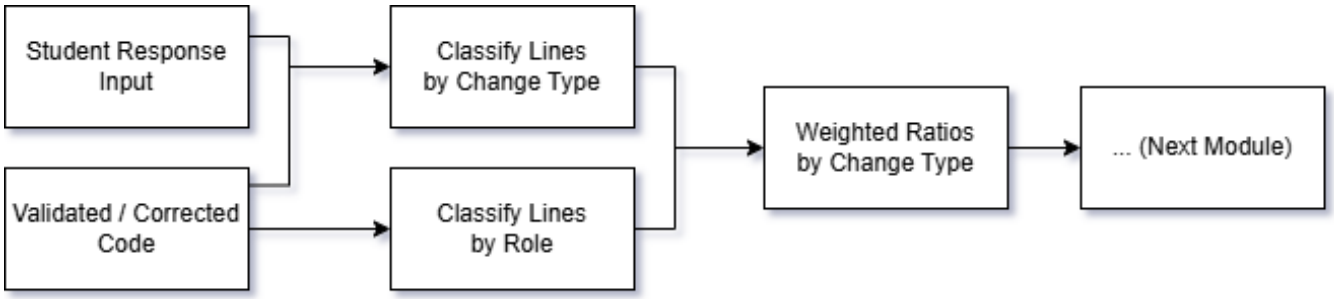

Fig 7. *The workflow of the Similarity and Evaluation Module.*

- The Grade Output Module:
  - In the final module, the ratios calculated from the previous module are used as input features for a machine learning model that predicts the final score of a code submission.

By extracting features from the LLM-corrected code relative to the original submission, this system outperforms the two previous frameworks, achieving a minimum MAE of 4.43%, depending on the regressor used.

	Model	MAE	RMSE	R^2 Score
1	Extra Trees Regressor	4.43	8.36	0.83
2	Random Forest Regressor	5.32	9.14	0.79
3	Extreme Gradient Boosting	4.90	9.03	0.78
4	Gradient Boosting Regressor	5.56	9.15	0.76
5	AdaBoost Regressor	6.27	10.26	0.74
6	K Neighbours Regressor	6.62	10.96	0.73
7	Decision Tree Regressor	5.55	10.29	0.73
8	Elastic Net	6.89	10.84	0.71
9	Lasso Least Angle Regression	7.23	11.33	0.70
10	Lass Regression	7.24	11.33	0.69

Fig 8. *The performance of different regression models in LLM-AC.*

Overall, although this framework is substantially more complex than the previous two, it achieves the best performance. Its final-grade MAE of 4.43% is lower than the roughly 5% MAE reported for each

grading rubric in StepGrade, which may accumulate in the final score, and the 6.5% MAE reported for CodEv, which may lead to worse performance if each rubric were weighted equally in the final grade. Moreover, by using machine learning, this framework implicitly captures how educators grade code, rather than relying on detailed prompts and an explicitly defined grading procedure.

For these reasons, this may be the best approach for the project. However, several limitations must still be considered. First, the project dataset does not include unit tests, and automating their generation using LLM may be difficult with Gemma 3 27B. As a result, the tests may need to be created manually or the unit-testing step may have to be removed entirely. Another important consideration is whether the coding capability of Gemma 3 27B is sufficient for this approach to work effectively.

References

[1] M. Akyash, K. Z. Azar, and H. Mardani Kamali, "StepGrade: Grading Programming Assignments with Context-Aware LLMs," in 2025 IEEE Integrated STEM Education Conference (ISEC), 2025, pp. 1–6, doi: 10.1109/ISEC64801.2025.11147374.

[2] E.-Q. Tseng, P.-C. Huang, C. Hsu, P.-Y. Wu, C.-T. Ku, and Y. Kang, "CodEv: An Automated Grading Framework Leveraging Large Language Models for Consistent and Constructive Feedback," in 2024 IEEE International Conference on Big Data (BigData), Washington, DC, USA, 2024, pp. 5442–5449, doi: 10.1109/BigData62323.2024.10825949.

[3] M. Mahdaoui, S. Nouh, M. S. El Kasmi Alaoui, and K. Kandali, "Automated Grading Method of Python Code Submissions Using Large Language Models and Machine Learning," Information, vol. 16, no. 8, Art. no. 674, 2025, doi: 10.3390/info16080674.